TEACHER'S TIPS

Designing and Training Your AlphaZero Agent

A field guide to catching silent bugs, spending compute wisely, and turning experiments into evidence.

Prepared for classroom use / Breakthrough Zero / 2026

Contents

> **The central lesson: a system that reveals why it failed will usually improve faster than one given more blind compute.**

1 Optimize for diagnosis, not training hours

AlphaZero projects fail quietly. The code runs, losses decrease, games finish, and the agent can still be learning the wrong game. In this setting, another five hours of training often makes a hidden bug more expensive rather than making the agent stronger.

> **Teacher's rule: make every phase capable of proving that the next phase is worth running.**

Before each major script, write a short review: what the algorithm currently does, which invariants must hold, where time is spent, and what evidence would justify the next change. Keep checkpoints, configurations, raw games, arena records, and failed runs. A regression that can be reproduced is an asset; a mysterious improvement is not yet knowledge.

The best student projects build a ladder of independently useful agents:

- Random player.
- Random-rollout MCTS.
- Simple alpha-beta baseline.
- PUCT with a dummy evaluator.
- PUCT after fixed-data pretraining.
- PUCT after online self-play.

Rate all of them. If a later rung is weaker, stop and diagnose it rather than changing three constants and trying again.

2 Choose a game and representation that expose mistakes

A good teaching game is small enough to run many controlled matches, but rich enough that search and learning both matter. Avoid games whose standard AlphaZero solution has been copied so often that implementation choices become invisible.

Write the rules twice. Keep one slow implementation that scans the board and states the rules literally. Optimize hot operations such as legal moves only beside that oracle, then compare them through thousands of seeded reachable positions.

Start with a smaller ruleset that exercises the same pipeline. For Breakthrough, a 5 x 5 board with one starting row makes complete games, searches, saved datasets, and overfitting experiments much faster. Do not build a disposable second engine: parameterize the shared rules and save the ruleset with every position. A phase graduates to 8 x 8 only after its mini version passes policy round trips, symmetry, terminal, replayability, and optimized-versus-literal rule tests.

When one fixed CNN/action shape is useful, a mini board can occupy an active region of the full tensor and treat padding as permanently illegal. Player 2 must rotate within the active board, not around the padded tensor's outer edge. Make standalone action transforms require an explicit ruleset so they cannot silently choose the wrong geometry.

For chess-like movement, the policy head deserves its own design. A useful pattern is a spatial source square plus mover-relative move planes. In Breakthrough this is only 8 x 8 x 3: forward-left, forward, and forward-right. Rotate Player 2 positions and actions by 180 degrees at the network boundary, while keeping the rules, search, logs, and saved data in absolute coordinates.

Required tests include:

- Every legal move has one unique policy index.
- Encode then decode returns the original absolute move for both players.
- Illegal output slots are masked before search.
- Every symmetry transforms state, policy, and value together.
- Optimized and reference legal moves agree on complete random games.
- Mini and target rulesets pass the same tests and data round trips.

3 Pick one value convention and make it impossible to violate

Use absolute values everywhere. Let `+1` mean Player 1 wins and `-1` mean Player 2 wins. Neural outputs, targets, node sums, Q-values, saved data, and arena logs all use that convention.

Do not negate values during backup. The only turn-dependent value operation is inside PUCT selection:

- Player 1 selects using `+Q + U`.
- Player 2 selects using `-Q + U`.

That is how one absolute Q-value supports both maximizing and minimizing. An unvisited child should use its parent's live Q as its initial Q. Test this on a hand-calculated tree before adding a network.

Mover-oriented input planes hide which absolute player is moving. If the value head must output an absolute value, add an identity plane such as `mover-is-Player-1`. Do not secretly create a relative latent value and multiply by a sign later; that simply relocates the most dangerous invariant.

> **If a value changes sign anywhere outside label-changing data augmentation or the `+Q`/`-Q` selection term, demand an explanation.**

4 Treat terminal states as a separate species

Alternating games invite a classic bug: code changes turns after every move, including the final move, and then evaluates the terminal position as if it belonged to the loser.

Choose and document one state-machine convention. In this project, an ordinary move changes turns; the terminal move does not. A finished state keeps the last mover and stores the absolute winner. Search checks terminal status before any evaluator call and backs up the rules result directly.

Test both colors and both terminal mechanisms:

- Reaching the goal edge.
- Leaving the opponent with no legal reply.
- A terminal node is never expanded or sent to the network.
- Make/unmake restores a position even when the move ended the game.
- A terminal child keeps the last mover rather than assuming alternation.

Terminal bugs frequently survive ordinary self-play because games still end. They are easiest to catch with tiny constructed positions and evaluator call counters.

5 Prove PUCT with a dummy network

Before Keras, use an evaluator whose policy gives every output slot equal weight and whose value is a random terminal rollout. Search must mask and renormalize legal actions itself. This separates four systems that are often debugged all at once: rules, search, data, and neural training.

The scalar PUCT reference should prove:

1. The caller's root state never changes.
2. Illegal priors never create children.
3. Visit counts add up exactly.
4. Backup never changes an absolute sign.
5. Player 1 prefers larger Q and Player 2 prefers smaller Q.
6. Parent-Q first-play urgency is exact.
7. Immediate wins are found for both colors.
8. Fixed seeds reproduce the same search.
9. Root noise changes search priors but not stored network priors.

Keep this simple implementation even after adding batching. It is the executable specification for the fast search.

6 Optimize measured hot paths, not attractive ideas

Profile on the actual game. Breakthrough bitboards make legal move generation simple, but a random rollout should not allocate every legal `Move` object just to choose one. Count set bits in the three target masks and instantiate only the selected move.

A tiny optional rollout rule can prefer an immediate win, then a capture, then any move. Benchmark it. In our local test the tactical selector was much faster than the old list-building path, but substantially slower than the optimized uniform selector and produced longer games. It remained an ablation, not the default.

State management is game-dependent too. Measure clone-and-replay, reset and replay, make/unmake, and cached node states. Here make/unmake was 10--18 percent slower because it doubled bitboard updates. Controlled end-to-end PUCT tests showed replay best at 32 simulations, while lazy state caching was 7--8 percent faster at 100 and 400. The chosen full-search path caches states only on visited nodes; a cheap-search variant is reconsidered only if profiling justifies it.

> **End-to-end throughput outranks a persuasive microbenchmark.**

7 Save self-play as a reusable scientific instrument

Self-play is the expensive part. Never save only already-encoded tensors. Keep unaugmented absolute games and transform them in the training loader.

For every searched position retain:

- Absolute bitboards, mover, ply, selected move, and game identity.
- Every legal absolute action.
- Network prior and possibly noised search prior separately.
- Per-action visits, absolute value sum, and squared-value sum.
- Root visits, value moments, initial evaluation, and greedy-leaf value.
- Full-search and sample-weight flags.
- Final absolute outcome and generation seed.

A checksummed manifest records rules, schema, model, MCTS configuration, code revision, and counts. Split train and validation by whole game. Apply the four exact symmetries in the loader, including value-sign changes when player labels are swapped.

This format lets a future student change the encoder, architecture, policy head, value target, or auxiliary head without paying for self-play again.

8 Use pretraining data as an offline laboratory

The final result is not the only plausible value target. It is simple and non-bootstrapped, but noisy. Search-derived targets such as root Q (soft-Z), a greedy child value, or a greedy backup may learn faster. Direct experiments on small Breakthrough found search-derived targets stronger than final-result training, but that is a starting hypothesis rather than a universal law.

Save enough statistics to reconstruct several targets. Then compare targets on the same fixed games, split, seeds, model budget, and wall-clock training time. Do the same for compact architectures, optimizer schedules, manual input features, and simple auxiliary heads.

Only winners from this fixed-data stage enter the online loop. This is cheaper and much easier to debug than changing architecture while the data distribution also moves.

Track both training fit and playing strength. A target that lowers validation loss can still produce a worse search agent, especially when the target itself contains search or network bias.

9 Understand root noise before choosing its constants

Root noise exists to prevent policy blind spots from becoming self-reinforcing. It is not value information and it is not automatically good. Add one Dirichlet sample to legal root priors before search; never add it to internal nodes, every simulation, or backed-up values.

Use total concentration divided across legal moves rather than copying a per-move alpha from Go or chess. Tune the noise fraction and concentration separately. Log raw/noised KL, visit entropy, low-prior moves explored, game diversity, missed tactics, and throughput.

Practical policy:

- No noise in deterministic tests.
- Usually no Dirichlet noise for uniform-policy rollout pretraining.
- Tuned noise in neural self-play.
- For evaluation, generate saved noisy opening prefixes of 5--10 plies, pair them with colors reversed, then make rated search noise-free.

Noise and move-selection temperature solve related but different problems. Do not change both in one experiment. Too little noise permits collapse; too much can consume a small search budget before it recovers obvious tactics.

10 Treat the replay buffer as a control system

A small active window is current but correlated. It can overfit and forget. A large window is diverse but stale; weak early play can slow learning long after the agent improves. There is no game-independent correct size.

Keep the complete immutable archive, but train from a bounded FIFO view over the newest complete games. Seed the online window with recent pretraining data and let new neural games evict it naturally. Log capacity, actual positions, wall-clock age, model age, policy surprise, unique-state redundancy, and the reuse factor: optimizer examples divided by new positions.

Choose candidate capacities only after measuring positions generated per HPC hour. Compare short, medium, and long windows at equal end-to-end wall time. Inspect both recent validation and an immutable anchor; either alone can reward the wrong behavior.

Avoid unlimited epochs merely because self-play is expensive. Reusing data is valuable until correlated old targets begin to overpower new evidence.

11 Evaluate strength fairly and make Elo honest

Iterations, simulations, and network evaluations do not cost the same across agents. Use equal wall-clock training budgets on the same hardware and equal wall-clock thinking time per move. Warm up imports, tracing, and devices before clocks start.

Deterministic agents need diverse games. Prepare opening prefixes with noise in the first 5--10 plies, save them before results are viewed, and play each opening twice with colors reversed. Record every move, time, node count, neural evaluation, seed, model hash, hardware description, and result.

Report wins, losses, draws, score, Elo difference, and a 95 percent confidence interval. Pool Elo is a convenient map; the paired head-to-head match is the evidence for an ablation. Never claim improvement from a point estimate whose uncertainty is larger than the effect that matters.

Anchor the pool with immutable agents: random, rollout MCTS, alpha-beta, pretrained PUCT, and selected old checkpoints. Negative results belong in the record.

12 Expect self-play to make the agent worse

A very common student result is:

```
rollout MCTS > pretrained PUCT > self-play PUCT
```

Do not hide it and do not immediately add heuristics. First evaluate four matched combinations: uniform policy plus rollout value, network policy plus rollout value, uniform policy plus network value, and the full network. This reveals whether the policy head, value head, or their interaction regressed.

Then check, in order:

1. Absolute signs and Player 2 selection direction.
2. Terminal-node handling.
3. Policy orientation, masks, and symmetry transforms.
4. Visit targets versus played one-hot moves.
5. Logit/probability and loss configuration.
6. Training/inference behavior of normalization layers.
7. Learning rate, reuse factor, and correlated small buffers.
8. Stale large buffers.
9. Exploration collapse or excessive noise.
10. Too few simulations and biased bootstrapped value targets.
11. Unequal evaluation compute or duplicate games.

Keep the pretrained checkpoint and every raw chunk immutable so the failure can be bisected instead of recreated.

13 Run research without building a pile of patches

Every variant needs a named hypothesis, one isolated change, a resolved configuration, and a fair wall-clock comparison. Record general findings in a separate conclusions file; keep the README focused on how the project works.

Useful experiments are simple enough to explain: value targets, playout-cap randomization, compact network size, global pooling, an opponent-next-policy head, rollout policy, root noise, or replay age. Complex modern techniques can wait until a measured bottleneck or failure gives them a job.

Report engineering observations separately from playing-strength conclusions. A faster move generator is not automatically a stronger agent, and a stronger fixed-simulation agent may be weaker per second. Repeat positive findings when the budget allows; label one-seed results preliminary.

The goal is not to reproduce an eight-year-old recipe. It is to create a small, legible experimental system in which a student can learn which ideas matter and why.

14 Stop/go checklist before an HPC run

Do not launch the expensive job until every answer below is yes.

- Do optimized rules match the literal oracle across seeded complete games?
- Do policy encode/decode and all four augmentations pass for both players?
- Is every stored and predicted value absolute Player 1 value?
- Are terminal nodes handled without an evaluator call?
- Does dummy PUCT pass visit, masking, FPU, sign, and immediate-win tests?
- Are network priors preserved separately from noised search priors?
- Can a self-play chunk be reloaded, checksummed, and retargeted?
- Are train/validation splits by game?
- Is a pure MCTS and alpha-beta baseline rated under equal time?
- Has a short profiler run identified the actual bottleneck?
- Are noise, replay, temperature, and search budgets explicit configuration?
- Will an interrupted worker lose at most one small chunk?
- Are seeds, code revision, hardware, and resolved configuration logged?
- Is there a regression ladder against pretrained and older anchors?

If not, spend the next hour making the problem observable. That is usually the highest-Elo use of the hour.

15 Further reading

- Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*.
- Wu, *Accelerating Self-Play Learning in Go* and KataGo's methods notes.
- Willemsen, Baier, and Kaisers, *Value Targets in Off-Policy AlphaZero*.
- Jones et al., *Scaling Scaling Laws with Board Games*.
- Tian et al., *ELF OpenGo: An Analysis and Open Reimplementation of AlphaZero*.
- OpenSpiel's AlphaZero implementation notes.
- AlphaZero.jl's small-game training guide and profiling utilities.
- Leela Chess Zero's encoder and retained training-data formats.

Read implementations for failure modes and engineering ideas, but demand a Breakthrough-specific reason and a fair ablation before adopting their constants.